Lab 1 实验报告

一、实验目的

在禁止控制流、比较运算符与常规算术运算的条件下，仅用位运算完成 19 个函数，训练三方面能力：**补码位级理解**（掩码构造、算术右移与逻辑右移的区别、进位与溢出的位级判定）、**IEEE 754 单精度浮点的位级操作**（符号 / 阶码 / 尾数的拆分重组、规格化与非规格化数的统一处理、round-to-nearest-even 舍入）、**受限范式下的算法设计**（用掩码替代分支，并满足每题的最大操作数上限）。

二、实验内容与约束

2.1 32 位整数机器模型

整数题统一采用 32 位位向量模型：

- `int` 为 32 位二进制补码；
- 对有符号数执行 `>>` 为**算术右移**，高位补符号位；
- 加法与左移按 32 位位向量解释，包括左移进入符号位的情况，超出 32 位的高位被丢弃；
- 移位量必须在 0–31 内。

生成最高位掩码、把字节移入最高字节、计算加法的低 32 位等写法都依赖上述模型。

2.2 整数题编码规则

允许：常量 `0`–`255`（含 `0x00`–`0xFF`）、函数参数与 `int` 局部变量、一元 `! ~`、二元 `& ^ | + << >>`。

禁止：`if` / `for` / `while` / `do` / `switch`；`&& || - * / % ?:`；全部比较运算符（`< <= > >= == !=`）；宏、额外函数、函数调用、强制类型转换；`int` 以外的类型（含 `unsigned`）；数组、结构体、联合体；超过该题最大操作数的实现。

> 注：赋值本身不计入操作数，但赋值右侧使用的运算符计数；负数常量会引入被禁用的负号，需用允许的运算构造所需位模式。P2 `bitXor` 进一步限制为**仅可用 `~` 和 `&`**。

2.3 浮点题编码规则

允许：`int` 与 `unsigned` 及其任意整数常量、整数算术 / 位运算 / 逻辑运算 / 比较运算、`if` / `while` / `for` 等控制流（P15–P17）。

仍禁止：浮点类型、浮点常量与浮点运算；强制类型转换；宏、额外函数与函数调用；`int`、`unsigned` 以外的类型；数组、结构体、联合体。

2.4 题目总览

编号	函数	输入约束	合法运算符	最大操作数	分值	核心手法
P1	`signMask()`	无参数	通用整数运算	2	1	左移构造最高位掩码
P2	`bitXor(x,y)`	任意 `int`	仅 `~ &`	8	2	德摩根律展开异或
P3	`negativePart(x)`	任意 `int`	通用整数运算	6	3	算术右移作条件掩码
P4	`copyByteWithin(x,src,dst)`	`src,dst` 为 0–3	通用整数运算	12	4	移位量由参数动态计算
P5	`logicalShift(x,n)`	`0<=n<=31`	通用整数运算	20	4	掩码清除符号扩展位
P6	`swapNibblePairs(x)`	任意 `int`	通用整数运算	18	4	半字节掩码 + 双向移位
P7	`secondLowestZeroBit(x)`	任意 `int`	通用整数运算	8	4	`~x & (x+1)` 定位最低 0 位
P8	`oddParity(x)`	任意 `int`	通用整数运算	56	5	折叠异或求奇偶
P9	`rotateRightBits(x,n)`	`n>=0`，按 32 取模	通用整数运算	16	5	逻辑右移与左移拼接
P10	`roundEvenPow2(x,n)`	`0<=x<=0x3fffffff`，`1<=n<=16`	通用整数运算	24	5	先四舍五入再修正偶数
P11	`midpointTowardFirst(x,y)`	任意 `int`	通用整数运算	32	5	无溢出中点 + 异号比较
P12	`isBetweenEitherOrder(x,a,b)`	任意 `int`	通用整数运算	48	7	无溢出 `<=` 谓词拼接
P13	`mul5Sat(x)`	任意 `int`	通用整数	30	7	绝对值与阈值比较后饱

编号	函数	输入约束	合法运算符	最大操作数	分值	核心手法
			运算			和
P14	`classifyAdd3(x,y,z)`	任意 `int`	通用整数运算	52	7	符号计数与进位计数比较
P15	`floatScaleThreeHalves(uf)`	任意单精度位模式	浮点题规则	60	7	尾数整数化 + 统一舍入
P16	`floatRoundEven(uf)`	任意单精度位模式	浮点题规则	65	10	整数位 + 余数位半程判断
P17	`float_i2f(x)`	任意 `int`	浮点题规则	40	10	找最高位后重排阶码尾数
P18	`bitCount(x)`	任意 `int`	通用整数运算	40	10	分治法并行计数
P19	`bitReverse(x)`	任意 `int`	通用整数运算	34	10	分治法块交换

三、逐题实现与分析

> 以下每题的"操作数"为按分值表口径的预估修正；**最终以 `./check_ops.py bits.c` 的输出为准**。

P1 `signMask()` — 最高位掩码

题意：返回仅最高有效位为 1 的掩码，即 `0x80000000`。

思路：在 32 位补码模型下，`1 << 31` 恰好把 1 移入符号位，位模式即 `0x80000000`。不需要任何条件判断。

实现

```
int signMask(void) {
  return 1 << 31;
}
```

边界：无输入，唯一确定。仅用 1 个运算符，远低于上限 2。

P2 `bitXor(x,y)` — 仅用 `~` 与 `&` 实现异或

题意：只允许 `~` `&`，实现 `x ^ y`。例：`bitXor(4,5)=1`，`bitXor(7,7)=0`。

思路：由真值表出发，`x ^ y = (x | y) & ~(x & y)`。再用德摩根律把 `|` 展开为 `~` 与 `&`：

```
x | y  =  ~(~x & ~y)
```

代回即得只用 `~`、`&` 的表达式。

实现

```
int bitXor(int x, int y) {
    return ~(~x & ~y) & ~(x & y);
}
```

操作数：`~x`、`~y`、`&`、`~`(外层)、`x&y`、`~`(外层)、`&` 共 7 个，满足上限 8。

边界：表达式对任意 32 位输入都成立，不依赖符号性或溢出，无需额外处理。

P3 `negativePart(x)` — 条件取负

题意：`x < 0` 时返回 `-x`，否则返回 0。例：`negativePart(-10)=10`，`negativePart(5)=0`。

思路：禁止比较运算符，因此用**算术右移**把符号位广播成整字掩码：

- `x >> 31` 在 `x < 0` 时为 `0xFFFFFFFF`，`x >= 0` 时为 `0`；
- 补码取负恒等式 `-x = ~x + 1`（对 `INT_MIN` 也成立）。

两者按位与即得结果：条件成立时输出 `-x`，否则与 0 相与被清零。

实现

```
int negativePart(int x){
  return (x >> 31) & (~x + 1);
}
```

边界：

- `x = 0`：掩码为 0，返回 0；同时 `~0+1 = 0`，两条路径一致。
- `x = INT_MIN`：`-INT_MIN` 在 32 位补码下溢出回 `INT_MIN`，掩码为全 1，返回 `0x80000000`，与题意（`-x` 的位模式）一致。
- `x = 5`：掩码为 0，返回 0。

操作数：`>>`、`~`、`+`、`&` 共 4 个，满足上限 6。

P4 `copyByteWithin(x,src,dst)` — 字节内复制

题意：把 `x` 的字节 `src` 复制到字节 `dst`，其余字节不变；字节编号 0（最低）到 3（最高），`src`、`dst` 均在 0–3。例：`copyByteWithin(0x11223344,0,2)=0x11443344`。

思路：动态移位量。因为 `src`、`dst` 是变量，字节位置需要按 `(src << 3)`、`(dst << 3)` 现场算出：

1. **取源字节**：`x >> (src << 3)` 把目标字节移到最低位，再 `& 0xFF` 留下 8 位（同时清除算术右移带来的符号扩展位）。
2. **清目标字节**：`~(0xFF << (dst << 3))` 生成"目标字节为 0、其余为 1"的掩码，与 `x` 相与。
3. **合并**：把源字节左移回 `dst` 位置后按位或。

实现

```
int copyByteWithin(int x, int src, int dst) {
  int src_byte = x >> (src << 3) & 0xFF;
  int dst_byte = ~(0xFF << (dst << 3)) & x;
  return dst_byte | src_byte << (dst << 3);
}
```

边界：

- `src = 3` 时 `dst << 3` 达到 24，`0xFF << 24` 恰好落在最高字节，不超出 0–31 的合法移位范围。
- 最高字节被取出时 `x >> 24` 会做符号扩展，但 `& 0xFF` 已经把高 24 位丢弃，因此负数输入同样正确。
- `src == dst` 时步骤 2、3 组合等价于原样写回，结果不变。

操作数：约 9 个，满足上限 12。

P5 `logicalShift(x,n)` — 逻辑右移

题意：把 `x` 逻辑右移 `n` 位，`0 <= n <= 31`。例：`logicalShift(0x87654321,4)=0x08765432`。

思路：C 语言的 `>>` 对 `int` 是算术右移，会在高 `n` 位补符号位。核心构造是**生成一个高 `n` 位为 0、其余为 1 的掩码**，用来抹掉符号扩展：

```
mask = ~( ((1 << 31) >> n) << 1 )
```

- `1 << 31` 得到 `0x80000000`；
- 算术右移 `n` 位得到"高 `n` 位为 1"的掩码；
- 再左移 1 位、取反，得到"高 `n` 位为 0、低位全 1"的掩码。

实现

```
int logicalShift(int x, int n) {
  int mask = ~(((1 << 31) >> n) << 1);
  return mask & (x >> n);
}
```

边界：

- `n = 0`：`(1<<31)>>0 = 0x80000000`，`<<1` 后为 `0x00000000`（最高位被丢出），取反得 `0xFFFFFFFF`，掩码全 1，返回 `x` 本身，正确。
- `n = 31`：掩码为 `~0xFFFFFFFF = 0`，结果只剩 `x >> 31` 的最低位（即原符号位），与逻辑右移 31 位一致。
- 掩码构造中的位移量始终在 0–31 内。

操作数：`<<`(1)、`>>`(n)、`<<`(1)、`~`(1)、`>>`(n)、`&`(1) 共 6 个，满足上限 20。

P6 `swapNibblePairs(x)` — 半字节两两互换

题意：把 `x` 每个字节内的高 4 位与低 4 位互换。例：`swapNibblePairs(0xAB)=0xBA`。

思路：构造"每字节低 4 位为 1"的掩码 `0x0F0F0F0F`，分别取出每一字节的低半字节与高半字节，再对调移位：

- `(x >> 4) & mask` 取各字节的高半字节并下移到低半字节位置；
- `(x & mask) << 4` 取各字节的低半字节并上移到高半字节位置；
- 两者按位或。

实现

```
int swapNibblePairs(int x) {
  int mask = 0x0F | (0x0F << 8) | (0x0F << 16) | (0x0F << 24);
  return ((x >> 4) & mask) | ((x & mask) << 4);
}
```

边界：

- 掩码由 4 个不超过 `0xFF` 的常量拼出，符合"常量不超过 255"的限制。
- `x >> 4` 是算术右移，最高字节的高 4 位会被填入符号位，但 `& mask` 只保留每字节的低 4 位，符号扩展位被彻底清除。
- 对 `x = 0xAB`：`(0xAB>>4)&0x0F = 0x0A`，`(0xAB&0x0F)<<4 = 0xB0`，或起来 `0xBA`，正确。

操作数：约 11 个，满足上限 18。

P7 `secondLowestZeroBit(x)` — 第二个最低 0 位

题意：返回标记"第二个最低的有效 0 位"的掩码。例：`secondLowestZeroBit(0xFFFFFFFA)=0x4`，`secondLowestZeroBit(0x7FFFFFFF)=0`，`secondLowestZeroBit(-1)=0`。

思路：利用恒等式

```
~x & (x + 1)   →   标记 x 中最低的那个 0 位
```

原理：`x + 1` 会把从最低 0 位开始的所有低位翻转为 0、该 0 位变 1（进位链），`~x` 只在该 0 位及更高位为 1，两者相与后只剩最低的 0 位。

求"第二个"的做法是**先把第一个 0 位补成 1**（`x |= first`），再对新的 `x` 重复同一运算。

实现

```
int secondLowestZeroBit(int x) {
  int first = ~x & (x + 1);
  x = x | first;
  int second = ~x & (x + 1);
  return second;
}
```

边界：

- `x = 0xFFFFFFFA`：最低 0 位是 bit0，`first = 0x1`；补 1 后最低 0 位变成 bit2，`second = 0x4`。
- `x = 0x7FFFFFFF`：唯一（最低）0 位是 bit31，补 1 后 `x = 0xFFFFFFFF`，`x+1` 溢出为 0、`~x = 0`，`second = 0`，与题例一致。
- `x = -1`（全 1）：`first = 0`，`x` 不变，`second = 0`，与题例一致。

操作数：`~`、`+`、`&`、`|`、`~`、`+`、`&` 共 7 个，满足上限 8。

P8 `oddParity(x)` — 奇校验位

题意：当 `x` 二进制表示中 1 的个数为**偶数**时返回 1，否则返回 0。例：`oddParity(5)=1`（`101` 有两个 1），`oddParity(7)=0`。

思路：**折叠异或（folding XOR）**。异或具有"归约奇偶性"的性质：把高 16 位与低 16 位异或、再对折 8 位、4 位、2 位、1 位，最终最低位即所有 32 位的异或和——它为 0 表示 1 的个数为偶数。

题目要求偶数个数返回 1，所以最后取反再取最低位。

实现

```
int oddParity(int x) {
  x = x ^ (x >> 16);
  x = x ^ (x >> 8);
  x = x ^ (x >> 4);
  x = x ^ (x >> 2);
  x = x ^ (x >> 1);
  return ~x & 1;
}
```

边界：

- `x = 5`：折叠后最低位为 0（偶）→ `~0 & 1 = 1`，正确。
- `x = 7`：折叠后最低位为 1（奇）→ `~1 & 1 = 0`，正确。

- `x = 0`：0 个 1（偶）→ 返回 1。
- 折叠过程中的 `>>` 是算术右移，高位补的是符号位，但外层 `^` 使得这些填充位不会影响最终归约结果（第 16 步已把高 16 位并入低位），且最终只取 bit0。

操作数：5 次折叠各 2 个 + `~`、`&` 共 12 个，满足上限 56。

P9 `rotateRightBits(x,n)` — 循环右移

题意：把 `x` 循环右移 `n` 位，`n >= 0`，实际位数按 32 取模。例：`rotateRightBits(0x12345678,8)=0x78123456`。

思路：循环右移 = **逻辑右移 `n` 位** 与 **左移 `32-n` 位** 的并集。三个要点：

1. `n = n & 31` 实现"按 32 取模"（也处理了 `n = 32` 这类输入）；
2. 右半部分复用 P5 的掩码技巧，得到真正的逻辑右移结果；
3. 左移量 `32 - n` 在 `n = 0` 时需要是 32，但移位量最大只允许 31。因此改用 `(~n + 1) & 31`，即 `(-n) mod 32`：`n = 0` 时为 0（不移，正确，因为此时右半部分已是原值），`n = 8` 时为 24。

实现

```
int rotateRightBits(int x, int n) {
  n = n & 31;
  int mask = ~(((1 << 31) >> n) << 1);
  int right = (x >> n) & mask;
  int left = x << ((~n + 1) & 31);
  return left | right;
}
```

边界：

- `n = 0`：`mask = 0xFFFFFFFF`，`right = x`；左移量为 0，`left = x`，`x | x = x`，正确。
- `n = 32`：`n & 31 = 0`，退化为 `n = 0`，符合"按 32 取模"。
- `n = 8`，`x = 0x12345678`：`right = 0x00123456`，`left = 0x78000000`，或起来 `0x78123456`，正确。

操作数：约 12 个，满足上限 16。

P10 `roundEvenPow2(x,n)` — 舍入到 2 的幂倍数（银行家舍入）

题意：把非负 `x` 舍入到最近的 `2^n` 的倍数；恰好落在两个倍数中间时，选择**商为偶数**的那个。`0 <= x <= 0x3fffffff`，`1 <= n <= 16`。例：`roundEvenPow2(10,2)=8`，`roundEvenPow2(14,2)=16`。

思路：本小题的关键是"先按普通四舍五入算，再对中点做偶数修正"。

1. `pow2 = 1 << n`，`low_mask = pow2 - 1`，余数 `r = x & low_mask`（等价 `x mod 2^n`）；
2. `half = pow2 >> 1` 是中点；
3. 先算 `y_prime = (x + half) >> n`，这是**标准四舍五入**的结果（余数 ≥ 半程即进位）；
4. 判断是否为中点：`is_half = !(r ^ half)`（用异或的零判等代替 `==`）；

5. 中点时，若原商 `y = x >> n` 为**奇数**，说明"进一位"恰好落到偶数商上，保持 `y_prime`；若原商为**偶数**，则 `y_prime = y + 1` 是奇数，需要减 1 回到偶数。即 `adjust = is_half & !(y & 1)`，最终 `result_y = y_prime - adjust`。

实现

```
int roundEvenPow2(int x, int n) {
  int pow2 = 1 << n;
  int low_mask = pow2 + ~0;
  int r = x & low_mask;
  int half = pow2 >> 1;
  int is_half = !(r ^ half);
  int y = x >> n;
  int y_odd = y & 1;
  int adjust = is_half & !y_odd;
  int y_prime = (x + half) >> n;
  int result_y = y_prime + (~adjust + 1);
  return result_y << n;
}
```

边界：

- `x = 10, n = 2`：`r = 2 = half` 为中点，`y = 2` 偶数 → `adjust = 1`，`y_prime = 3`，修正为 2，结果 8。
- `x = 14, n = 2`：中点，`y = 3` 奇数 → `adjust = 0`，`y_prime = 4`，结果 16。
- `x = 12, n = 2`：非中点 (`r = 0 < half`)，`y_prime = 14 >> 2 = 3`，结果 12，无舍入。
- `x = 0`：任何 `n` 下 `r = 0`、`y_prime = 0`，结果 0。
- 由于 `x <= 0x3fffffff` 且 `n >= 1`，`x + half <= 0x3fffffff + 0x8000`，不会触及符号位，加法无溢出风险。

操作数：约 17 个，满足上限 24。

P11 `midpointTowardFirst(x,y)` — 无溢出的中点（奇偶取近）

题意：返回精确数学中点 `(x+y)/2`，不允许溢出；若中点恰在两个整数之间，取**离第一个参数 `x` 更近**的那个。例：`midpointTowardFirst(4,7)=5`，`midpointTowardFirst(7,4)=6`。

思路：分三步。

1. **无溢出求下取整中点**：直接 `(x+y)>>1` 可能溢出，改用 `mid = (x & y) + ((x ^ y) >> 1)`， 这是经典的"平均不溢出"恒等式（把共同的位与不同的位分开处理）。
2. **判奇**：`x + y` 为奇数 ⇔ `x`、`y` 最低位不同 ⇔ `is_odd = (x ^ y) & 1`。此时真实中点带 `.5`，需要决定往上取还是往下取。
3. **判断 `x > y`**：不能直接用比较运算符，改为按符号组合判断：
   - 异号（`diff_sign = sign_x ^ sign_y`）时，`x >= y` 等价于 `x >= 0`，即 `sign_x == 0`；
   - 同号时做差 `x - y` 不溢出，`x >= y` 等价于 `(x - y) >> 31 == 0`。

综合得 x_ge_y。当 x > y 时中点在 mid 与 mid+1 之间，离 x 更近的是 mid + 1；反之取 mid。

实现

```
int midpointTowardFirst(int x, int y) {
  int mid = (x & y) + ((x ^ y) >> 1);
  int is_odd = (x ^ y) & 1;

  int sign_x = x >> 31;
  int sign_y = y >> 31;
  int diff_sign = sign_x ^ sign_y;

  int x_minus_y = x + ~y + 1;
  int same_sign_ge = !(x_minus_y >> 31);
  int diff_sign_ge = !sign_x;

  int x_ge_y = (diff_sign & diff_sign_ge) | (~diff_sign & same_sign_ge);

  return mid + (is_odd & x_ge_y);
}
```

边界：

- (4,7)：is_odd = 1，x < y → x_ge_y = 0，mid = (4&7) + ((4^7)>>1) = 4+1 = 5，返回 5。
- (7,4)：x_ge_y = 1，mid = 5，返回 6。
- (x, y) = (INT_MAX, INT_MAX)：mid = INT_MAX，is_odd = 0，返回 INT_MAX，全程无溢出。
- (INT_MAX, INT_MIN)：异号，x_ge_y = 1；mid = (0x7FFFFFFF & 0x80000000) + ((0xFFFFFFFF) >> 1) = 0 + (-1) = -1，is_odd = 1，返回 0，即 (-1 + 2^31-1...) 的正确中点。
- 同号做差不会溢出，因此 x_minus_y 的符号位可信。

操作数：约 21 个，满足上限 32。

P12 isBetweenEitherOrder(x,a,b) — 区间判定（端点无序）

题意：x 落于以 a、b 为端点的闭区间内（端点顺序任意）时返回 1。例：isBetweenEitherOrder(5,8,3)=1。

思路：区间判定等价于两个方向的合取或：

```
(a <= x && x <= b) || (b <= x && x <= a)
```

难点在于**在不使用比较运算符、且不溢出**的前提下实现 u <= v。做法沿用 P11 的策略：先看符号是否相同。

- 异号（su ^ sv == 1）：u <= v 等价于 u < 0，即符号位 su 为 1；
- 同号：做差 u - v 不会溢出，u <= v 等价于 (u - v) >> 31 为 1 **或** 两者相等。

四个方向各构造一个谓词 le_ax、le_xb、le_bx、le_xa，最后 (le_ax & le_xb) | (le_bx & le_xa)。

实现

```
int isBetweenEitherOrder(int x, int a, int b) {
  int sa = a >> 31;
  int sb = b >> 31;
  int sx = x >> 31;

  int nx = ~x;

  int da = a + nx + 1;       // a - x
  int db = b + nx + 1;       // b - x
  int dxa = ~da + 1;         // x - a
  int dxb = ~db + 1;         // x - b

  int dax = sa ^ sx;
  int dbx = sb ^ sx;

  int eq_ax = !(a ^ x);
  int eq_bx = !(b ^ x);

  int le_ax = (dax & sa) | (~dax & ((da >> 31) | eq_ax));
  int le_bx = (dbx & sb) | (~dbx & ((db >> 31) | eq_bx));
  int le_xa = (dax & sx) | (~dax & ((dxa >> 31) | eq_ax));
  int le_xb = (dbx & sx) | (~dbx & ((dxb >> 31) | eq_bx));

  return ((le_ax & le_xb) | (le_bx & le_xa)) & 1;
}
```

边界：

- 端点相等（`a == b`）：`(a<=x && x<=a) || (b<=x && x<=b)` 都退化为 `x == a`，判定仍然正确。
- `x = INT_MIN`、`a = INT_MIN`：`a - x = 0`，`eq_ax = 1`，`le_ax = 1`（含端点）。
- `a`、`b` 异号且跨越 0 的区间：差值不会溢出（同号才做差），符号组合判断覆盖。
- 返回值用 `& 1` 归一化为 0/1。

操作数：约 40 个，满足上限 48。

P13 `mul5Sat(x)` — 乘 5 的饱和处理

题意：返回 `x*5`；若溢出则返回 `INT_MAX(0x7fffffff)` 或 `INT_MIN(0x80000000)`。例：`mul5Sat(1)=0x5`，`mul5Sat(0x40000000)=0x7fffffff`。

思路：禁止减法与比较，因此需要：

1. **取绝对值**：`sign = x >> 31`，`abs_x = (x + sign) ^ sign`（经典无分支绝对值，`INT_MIN` 也成立）。
2. **阈值比较**：`|x| * 5 > INT_MAX` ⇔ `|x| > INT_MAX / 5`，而 `INT_MAX / 5 = 0x19999999 = 429496729`。构造常量 `thresh = 0x19999999`（由 4 个 ≤ `0xFF` 的常量拼出），比较 `abs_x` 与 `thresh`

的大小用差值符号位实现：`diff = abs_x - thresh`，`overflow = !((diff >> 31) | !diff)`（差值既非负又不为零 ⇔ 严格大于）。

3. **饱和值选择**：`sat = sign ? INT_MIN : INT_MAX`，用符号掩码无分支选择。
4. **正常结果**：`x*5 = (x << 2) + x`。
5. **最终选择**：用 `overflow` 展开成全 1/全 0 掩码在 `sat` 与 `result` 之间二选一。

实现

```
int mul5Sat(int x) {
  int sign = x >> 31;
  int abs_x = (x + sign) ^ sign;
  int thresh = (0x19 << 24) | (0x99 << 16) | (0x99 << 8) | 0x99; // 0x19999999
  int diff = abs_x + ~thresh + 1;
  int overflow = !((diff >> 31) | (!diff));

  int sign_bit = 1 << 31;
  int max = ~sign_bit;
  int min = sign_bit;
  int sat = (sign & min) | (~sign & max);

  int result = (x << 2) + x;
  int ov_mask = ~overflow + 1;
  return (ov_mask & sat) | (~ov_mask & result);
}
```

边界：

- `x = 1`：`abs_x = 1 <= thresh`，不溢出，`result = 5`。
- `x = 0x40000000`：`abs_x = 1073741824 > thresh` → 溢出，`sign = 0` → 返回 `INT_MAX`。
- 正负对称性：`|x| > INT_MAX/5` 时，正方向 `5|x| > INT_MAX`，负方向 `5|x| > 2^31`，两条边界相差 1 个 ULP，**同一阈值即可同时覆盖**，这正是阈值取 `INT_MAX/5` 而非 `(INT_MAX+1)/5` 的原因。
- `x = INT_MIN`：`abs_x = 0x80000000`（作为无符号看是 2^31），远大于阈值，`sign = -1` → 返回 `INT_MIN`。
- `x = 0`：`result = 0`，不溢出。
- 阈值构造的 4 个常量均不超过 `0xFF`。

操作数：恰好 30 个，等于上限，无冗余空间。

P14 `classifyAdd3(x,y,z)` — 三数之和的分类

题意：判断精确数学和 `x+y+z`：大于 `INT_MAX` 返回 1，小于 `INT_MIN` 返回 -1，否则返回 0。禁止使用更宽类型。

思路：把三数求和拆成**两次 32 位加法**，同时统计两个量：

- `K = kx + ky + kz`：三个加数中**负数（符号位为 1）的个数**，取值 0–3；
- `C = c1 + c2`：两次加法实际产生的**进位次数**（每次进位用无溢出判定式 `(a&b) | ((a|b) & ~s)` 的符号位提取）。

令 `D = C - K`。两者之差反映"净进位/净借位"：

- `D > 0`：净进位 → 和超过 `INT_MAX` → 返回 1；
- `D < 0`：净借位 → 和低于 `INT_MIN` → 返回 -1；
- `D == 0`：无净溢出，此时再看中间和 `s2` 的符号位——若 `D == 0` 而 `s2` 为负，说明三个同号正数相加后符号翻转（进位被符号位抵消），仍属正向溢出。

分类结果通过 `pos`、`neg` 两个 0/1 标志相加得到：`return pos + (~neg + 1)`（即 `pos - neg`）。

实现

```
int classifyAdd3(int x, int y, int z) {
  int kx = (x >> 31) & 1;
  int ky = (y >> 31) & 1;
  int kz = (z >> 31) & 1;
  int K = kx + ky + kz;

  int s1 = x + y;
  int c1 = ((x & y) | ((x | y) & ~s1)) >> 31 & 1;
  int s2 = s1 + z;
  int c2 = ((s1 & z) | ((s1 | z) & ~s2)) >> 31 & 1;
  int C = c1 + c2;

  int D = C + (~K + 1);
  int D_neg = D >> 31;
  int D_zero = !D;
  int D_pos = !D_neg & !D_zero;

  int s2_sign = (s2 >> 31) & 1;

  int pos = D_pos | (D_zero & s2_sign);
  int D_plus_1 = D + 1;
  int D_is_minus_1 = !D_plus_1;
  int D_le_minus_2 = D_neg & !D_is_minus_1;
  int neg = D_le_minus_2 | (D_is_minus_1 & !s2_sign);

  return pos + (~neg + 1);
}
```

边界：

- `x = y = z = 0x55555555`：三正数，`K = 0`；`x+y = 0xAAAAAAAA` 但在 `c1` 的判定式中 `(x&y)|((x|y)&~s1)` 未产生符号位，`c1 = 0`；`s2 = 0xFFFFFFFF`（负），`c2 = 0`；`C = 0`，`D = 0` → 走 `D_zero & s2_sign = 1` 分支，返回 1，正确（真实和 `0xFFFFFFFF` `F > INT_MAX`）。
- 三数同为正但和不溢出：`s2` 符号位为 0，`pos = 0`，返回 0。
- 三个 `INT_MIN`：`K = 3`，进位使 `C` 增大，`D < 0`，`neg = 1`，返回 -1。
- `D == -1` 这一档单独处理（`D_is_minus_1 & !s2_sign`），用于区分"恰好落到 `INT_MIN` 边界"与"低于 `INT_MIN`"。
- 返回值 `pos - neg` 恰好是 1 / 0 / -1。

操作数：约 45 个，满足上限 52。

P15 `floatScaleThreeHalves(uf)` — 计算 `f * 3 / 2`

题意：把 `uf` 按 IEEE 754 单精度解读，返回 `f * 1.5` 的位模式；使用 round-to-nearest-even；保留 `+0` / `-0` 的符号；`NaN` 原样返回。

思路：把浮点乘法转化为**尾数整数乘法 + 阶码重算**，从而完全避开浮点运算：

1. 拆出 `sign`、`exp`、`frac`；`exp == 255`（Inf/NaN）与 `±0` 直接返回。
2. 统一得到有效尾数 `mant` 与真实阶 `E`：规格化数 `mant = (1<<23)|frac`、`E = exp - 127`；非规格化数 `mant = frac`、`E = -126`。
3. 乘以 3/2 等价于先算 `V = mant * 3`（整数运算，`mant < 2^24`，`3*mant < 2^26`，不溢出）；然后从 `V` 的最高有效位 `p` 反推新的阶码： `E_new = E + p - 24`（因为归一化后尾数隐含位在 `p`，小数位共 23 位）。
4. 舍入：取 `high = V >> (p - 23)` 作为新的 23 位尾数位段，被丢弃的 `dropped` 与半程 `half` 比较，采用 **round-half-to-even**（`dropped > half` 进位，`dropped == half` 时看尾数最低位）；
5. 尾数进位若溢出到 `0x800000`，则尾数归零、阶码加一；阶码达到 255 时返回对应符号的无穷。
6. `exp_new <= 0` 时进入非规格化区间，用 `(V>>1) + ((V&1)&((V>>1)&1))` 实现一次右移 1 位的偶数舍入。

实现

```
unsigned floatScaleThreeHalves(unsigned uf) {
    unsigned sign = uf & 0x80000000;
    unsigned exp = (uf >> 23) & 0xFF;
    unsigned frac = uf & 0x7FFFFF;

    if (exp == 255) {
        return uf;
    }

    if (exp == 0 && frac == 0) {
        return uf;
    }

    unsigned mant;
    int E;
    if (exp == 0) {
        mant = frac;
        E = -126;
    } else {
        mant = (1 << 23) | frac;
        E = exp - 127;
    }

    unsigned V = mant * 3;

    int p = 31;
    while (p >= 0 && !(V & (1u << p))) {
```

```
            p--;
        }

        int E_new = E + p - 24;
        int exp_new = E_new + 127;

        if (exp_new >= 255) {
            return sign | 0x7F800000;
        }

        unsigned frac_new;
        if (exp_new > 0) {
            int shift = p - 23;
            unsigned high = V >> shift;
            frac_new = high & 0x7FFFFF;

            if (shift > 0) {
                unsigned dropped = V & ((1u << shift) - 1);
                unsigned half = 1u << (shift - 1);
                if (dropped > half || (dropped == half && (frac_new & 1))) {
                    frac_new++;
                    if (frac_new == 0x800000) {
                        frac_new = 0;
                        exp_new++;
                        if (exp_new >= 255) {
                            return sign | 0x7F800000;
                        }
                    }
                }
            }
        } else {
            frac_new = (V >> 1) + ((V & 1) & ((V >> 1) & 1));
            exp_new = 0;
        }

        return sign | (exp_new << 23) | frac_new;
    }
```

边界：

- `uf = 1.0f`（`0x3F800000`）：`mant = 0x800000`，`E = 0`，`V = 0x1800000`，`p = 24` → `E_new = 0`，`shift = 1`，`high = 0xC00000`，尾数取 `0x400000`，结果 `1.5f`，正确。
- `NaN`（`exp == 255`）：直接返回原 `uf`，保留 payload。
- `±0`（`exp == 0 && frac == 0`）：直接返回原 `uf`，`-0` 的符号保留。
- **非规格化输入**：`exp == 0` 但 `frac != 0` 时按 `mant = frac, E = -126` 进入统一流程；结果若不是非规格化区间则由 `exp_new` 判断走规格化分支，否则走 `exp_new <= 0` 的舍入分支。
- **溢出**：`f` 接近最大有限数时 `exp_new >= 255`，返回 `sign | 0x7F800000`（带符号的无穷）；尾数进位导致的二次溢出也做了同样处理。
- **下溢**：`exp_new == 0` 时用整数移位实现"再舍入一次"，保证非规格化区的 half-even 行为。

操作数：约 50 个，满足上限 60。

P16 `floatRoundEven(uf)` — 四舍五入到整数（银行家舍入）

题意：把 `uf` 表示的浮点值舍入到最近的整数（`.5` 取偶），返回该整数的单精度浮点位模式；结果为 0 时保留输入符号（如 `-0.3 → -0`）；`NaN` / `Inf` 原样返回。

思路：按数量级分段处理，避免无谓的位操作。

1. `exp == 255`（NaN/Inf）与 `±0`：原样返回。
2. **`exp < 126`**（`|f| < 0.5`）：结果必为 0，直接返回 `sign`（保留 `-0` 的符号）。
3. **`exp == 126`**（`0.5 <= |f| < 1`）：`frac == 0` 表示恰好 `±0.5`，按银行家舍入取偶数 0，返回 `sign`；否则 `|f| > 0.5`，返回 `sign | 0x3F800000`（即 `±1.0`）。
4. **`E >= 23`**：尾数 23 位全是整数部分，值本身已是整数，返回 `uf`。
5. 其余情况：取 `M = (1<<23)|frac`，`shift = 23 - E`，整数部分 `I = M >> shift`，小数余数 `F = M & ((1<<shift)-1)`，半程 `half = 1 << (shift-1)`。按 round-half-to-even 决定是否 `I++`。
6. **重新归一化**：找 `I` 的最高位 `p`。`p == 23` 表示 `I` 恰好进位到 `2^23`（23 位全 1 加一），此时结果就是 `2^23`，直接返回 `sign | (150 << 23)`（阶码 `127+23 = 150`）；否则 `new_exp = 127 + p`，`new_frac = (I << (23-p)) & 0x7FFFFF`。

实现

```
unsigned floatRoundEven(unsigned uf) {
    unsigned sign = uf & 0x80000000;
    unsigned exp = (uf >> 23) & 0xFF;
    unsigned frac = uf & 0x7FFFFF;

    if (exp == 255) return uf;
    if (exp == 0 && frac == 0) return uf;

    if (exp < 127) {
        if (exp < 126) {
            return sign;
        } else {
            if (frac == 0) {
                return sign;
            } else {
                return sign | 0x3F800000;
            }
        }
    }

    int E = exp - 127;
    if (E >= 23) return uf;

    unsigned M = (1 << 23) | frac;
    int shift = 23 - E;
    unsigned I = M >> shift;
    unsigned F = M & ((1 << shift) - 1);
    unsigned half = 1 << (shift - 1);

    if (F > half || (F == half && (I & 1))) {
        I++;
```

```
    }

    int p = 31;
    while (p >= 0 && !(I & (1u << p))) {
        p--;
    }

    if (p == 23) {
        return sign | (150 << 23);
    } else {
        unsigned new_exp = 127 + p;
        unsigned new_frac = (I << (23 - p)) & 0x7FFFFF;
        return sign | (new_exp << 23) | new_frac;
    }
}
```

边界：

- ±0.5（0x3F000000 / 0xBF000000）：exp = 126、frac = 0 → 返回 ±0，符合".5 取偶"。
- 0.5000001：exp = 126、frac != 0 → 返回 1.0。
- -0.3：exp = 125 < 126 → 返回 0x80000000（-0），符号保留。
- -2.5：E = 1，shift = 22，I = 2（奇数）、F == half → I++ 得 3；重新归一化得 3.0。+2.5 → 2.0、-2.5 → -2.0 亦同理（I = 2 偶数 → 不进位）。
- 极大值（E >= 23）：本身是整数，原样返回，不会进入移位逻辑。
- 负数由 sign 单独保管，全部计算在绝对值域进行，因此无需处理补码符号问题。

操作数：约 55 个，满足上限 65。

P17 float_i2f(x) — 整数转单精度浮点

题意：返回 (float) x 的位模式（以 unsigned 承载），禁止使用浮点类型与强制转换。

思路：

1. x == 0 直接返回 0（+0）。
2. 取符号 sign = x & 0x80000000；求绝对值：x < 0 时 abs_x = ~x + 1（处理 INT_MIN 时不溢出）。
3. **找最高有效位**：循环右移 abs_x 直到剩 1，计数得 e，这是真实的二进制阶。
4. exp = e + 127。
5. **尾数**：
    - e <= 23：直接左移对齐到 23 位，mantissa = (abs_x << (23-e)) & 0x7FFFFF；
    - e > 23：需要右移丢弃 shift = e - 23 位，取出高位作为尾数，低位 dropped 与半程比较，按 **round-half-to-even** 决定是否进位；尾数进位到 0x800000 时归零并 exp++。

实现

```
unsigned float_i2f(int x) {
  if (x == 0) return 0;

  unsigned sign = x & 0x80000000;
```

```
  unsigned ux = x;
  unsigned abs_x;
  if (x < 0) {
    abs_x = ~ux + 1;
  } else {
    abs_x = ux;
  }

  int e = 0;
  unsigned temp = abs_x;
  while (temp > 1) {
    temp >>= 1;
    e++;
  }

  unsigned exp = e + 127;
  unsigned mantissa;

  if (e <= 23) {
    mantissa = (abs_x << (23 - e)) & 0x7FFFFF;
  } else {
    int shift = e - 23;
    unsigned high = abs_x >> shift;
    unsigned m = high & 0x7FFFFF;
    unsigned mask = (1 << shift) - 1;
    unsigned dropped = abs_x & mask;
    unsigned half = 1 << (shift - 1);

    if (dropped > half || (dropped == half && (m & 1))) {
      m++;
      if (m == 0x800000) {
        m = 0;
        exp++;
      }
    }
    mantissa = m;
  }

  return sign | (exp << 23) | mantissa;
}
```

边界：

- `x = 0`：显式返回 `+0`（注意不能走后面的流程，否则 `e` 无意义）。
- `x = INT_MIN`（`0x80000000`）：`abs_x = ~0x80000000 + 1 = 0x80000000`，`e = 31`，`exp = 158`，`shift = 8`，`dropped = 0`，结果 `0xCF000000`，即 `-2^31`，正确。
- `x = 1`：`e = 0`，`mantissa = 1 << 23 & 0x7FFFFF = 0`，返回 `0x3F800000 = 1.0f`。
- `x = 0x7FFFFFFF`（`2^31 - 1`）：`e = 30`，`shift = 7`，`dropped = 0x7F > half = 0x40` → 进位，尾数变 `0x7FFFFF + 1 = 0x800000` → 尾数归零、`exp` 加一，得 `0x4F000000 = 2^31`，与浮点舍入结果一致。
- 半程情形（`dropped == half`）由尾数最低位决定是否进位，避免"总是进位"的偏差。

操作数：约 35 个，满足上限 40。

P18 `bitCount(x)` — 统计 1 的个数

题意：返回 `x` 二进制中 1 的个数。例：`bitCount(5)=2`，`bitCount(7)=3`。

思路：**分治并行计数**（SWAR）。先在 2 位域内统计、再在 4 位、8 位、16 位域内合并，`log2(32) = 5` 步完成。

掩码由小常量拼接生成（受"常量 ≤ 255"约束）：

- `0x55555555`：每 2 位取低位；
- `0x33333333`：每 4 位取低 2 位；
- `0x0F0F0F0F`：每 8 位取低 4 位；
- `0x00FF00FF`、`0x0000FFFF`：逐级合并。

每一步 `(count & mask) + ((count >> k) & mask)` 把相邻两个域的和放进同一个域，域宽翻倍，互不干扰。

实现

```
int bitCount(int x) {
  int m1 = 0x55 | (0x55 << 8);      // 0x00005555
  m1 = m1 | (m1 << 16);             // 0x55555555

  int m2 = 0x33 | (0x33 << 8);      // 0x00003333
  m2 = m2 | (m2 << 16);             // 0x33333333

  int m4 = 0x0F | (0x0F << 8);      // 0x00000F0F
  m4 = m4 | (m4 << 16);             // 0x0F0F0F0F

  int m8 = 0xFF | (0xFF << 16);     // 0x00FF00FF
  int m16 = 0xFF | (0xFF << 8);     // 0x0000FFFF

  int count = (x & m1) + ((x >> 1) & m1);
  count = (count & m2) + ((count >> 2) & m2);
  count = (count + (count >> 4)) & m4;
  count = (count + (count >> 8)) & m8;
  count = (count + (count >> 16)) & m16;
  return count;
}
```

边界：

- `x = -1`（32 个 1）：每步域宽翻倍，末步得 `32`，正确。
- `x = 0`：全 0，返回 0。
- `x = INT_MIN`：只有 bit31 为 1，返回 1。
- 第 3–5 步不必再先 `&` 再移，因为此时每个域的和不会超出域宽（4 位域最大和 4，8 位域最大 8，16 位域最大 16），加法不会串域；最后一步再掩码截取，节省操作数。

- 中间值均为小正数，不涉及符号问题；`x` 本身为负数时第一步用的是 `x & m1` 与 `(x >> 1) & m1`，算术右移的填充位被掩码清除，不影响结果。

操作数：约 34 个，满足上限 40。

P19 `bitReverse(x)` — 位反转

题意：把 32 位整数的位序完全颠倒。例：`bitReverse(0x80000004)=0x20000001`，`bitReverse(0x7FFFFFFF)=0xFFFFFFFE`。

思路：**分治块交换**，与 P18 的分治计数互为对偶。依次交换 16 位块、8 位块、4 位块、2 位块、1 位块，`log2(32) = 5` 步完成。

掩码用递推方式生成，节省操作数：

```
mask = 0x0000FFFF                       // 交换高/低 16 位
mask = mask ^ (mask << 8) = 0x00FF00FF  // 交换 8 位块
mask = mask ^ (mask << 4) = 0x0F0F0F0F  // 交换 4 位块
mask = mask ^ (mask << 2) = 0x33333333  // 交换 2 位块
mask = mask ^ (mask << 1) = 0x55555555  // 交换 1 位块
```

每一步用同一套模板完成"交换相邻两块"：

```
x = ((x >> k) & mask) | ((x & mask) << k)
```

其中 `mask` 是"每 `2k` 位中低 `k` 位为 1"的掩码：右移 `k` 后取 `mask` 位得到原来高块的位并下移，`& mask` 后左移 `k` 把原来低块的位送上去，两者或起来即完成交换。

实现

```
int bitReverse(int x)
{
  int mask = (0xFF << 8) | 0xFF;

  x = ((x >> 16) & mask) | (x << 16);

  mask = mask ^ (mask << 8);
  x = ((x >> 8) & mask) | ((x & mask) << 8);

  mask = mask ^ (mask << 4);
  x = ((x >> 4) & mask) | ((x & mask) << 4);

  mask = mask ^ (mask << 2);
  x = ((x >> 2) & mask) | ((x & mask) << 2);

  mask = mask ^ (mask << 1);
  x = ((x >> 1) & mask) | ((x & mask) << 1);
```

```
  return x;
}
```

边界：

- 第一步 `x << 16` 与 `x >> 16` 是半字交换，无需掩码（`x << 16` 自然丢弃高 16 位、`x >> 16` 需要 `& mask` 清掉算术右移的符号扩展位，这里的 `mask = 0x0000FFFF` 正好承担该职责）。
- `x = 0x80000004`：交换后应为 `0x20000001`，与题例一致。
- `x = -1`：全 1 的位模式在任何块交换下不变，返回 `-1`。
- 每步的 `<< k` 均在 0–31 范围内，块交换不产生溢出。

操作数：约 28 个，满足上限 34。

四、测试与验证

4.1 构建

```
make clean
make all
```

> `btest` **不会**在源文件修改后自动重编译，每次改动 `bits.c` 后都必须重新执行上述两步，否则测到的可能是旧代码。

4.2 规则检查（运算符与操作数）

```
./check_ops.py bits.c
```

期望输出末行：

```
All 19 functions passed operator checks.
```

> 不要直接用 `./dlc bits.c` 检查整份文件——仓库自带的 `dlc` 内置的是旧函数名，`check_ops.py` 会为新题逐题选择同规则的代理函数名再调用 `dlc`，代理过程不会放宽合法运算符。

4.3 单题调试

```
./btest -f 函数名
./btest -f roundEvenPow2 -1 5 -2 1
./btest -f copyByteWithin -1 0x11223344 -2 0 -3 2
```

失败时会打印输入、实际结果与期望结果，便于定位问题类型（符号 / 溢出 / 移位 / 舍入 / 浮点特殊值）。

4.4 完整测试

```
./btest          # 全部 19 题，期望 Total points: 110/110
./test.sh        # 一步完成构建 + 规则检查 + 完整测试，满分时返回 0
```

4.5 实测结果

编号	函数	规则检查（操作数/上限）	错误数	得分
P1	signMask	/ 2		1
P2	bitXor	/ 8		2
P3	negativePart	/ 6		3
P4	copyByteWithin	/ 12		4
P5	logicalShift	/ 20		4
P6	swapNibblePairs	/ 18		4
P7	secondLowestZeroBit	/ 8		4
P8	oddParity	/ 56		5
P9	rotateRightBits	/ 16		5
P10	roundEvenPow2	/ 24		5
P11	midpointTowardFirst	/ 32		5
P12	isBetweenEitherOrder	/ 48		7
P13	mul5Sat	/ 30		7
P14	classifyAdd3	/ 52		7
P15	floatScaleThreeHalves	/ 60		7
P16	floatRoundEven	/ 65		10
P17	float_i2f	/ 40		10
P18	bitCount	/ 40		10
P19	bitReverse	/ 34		10
—	**合计**	—	—	**110**

> 本表"操作数"与"错误数"两列留空，请在本机运行 `./check_ops.py bits.c` 与 `./btest` 后按实际输出填写，作为本报告的实测证据。

五、关键总结

5.1 不用比较运算符实现条件判断

整数题全部禁用 `if` 与比较运算符，条件判断统一抽象为两种模式：

模式	表达式	用途
符号广播	`x >> 31`	得到全 1 / 全 0 掩码，作无分支选择器（P3、P13）
零判等	`!(a ^ b)`	以异或的零判等代替 `==`（P10、P12、P14）
差值符号	`(a - b) >> 31`	同号比较大小（P11、P12、P13）
符号异或	`(a >> 31) ^ (b >> 31)`	异号时改判符号位（P11、P12）

同号才可安全做差是本实验反复出现的前提：异号相减会溢出使符号位失真，必须切换到"看符号位"的分支。

5.2 无溢出算术的三个恒等式

- 绝对值：`abs(x) = (x + (x >> 31)) ^ (x >> 31)`；
- 取负：`-x = ~x + 1`；
- 平均：`(x & y) + ((x ^ y) >> 1)`。

三者在 `INT_MIN` / `INT_MAX` 处均不产生未定义行为，是本实验处理边界的基础工具。

5.3 掩码构造

- **广播掩码**：`1 << 31` → 最高位；`~(((1 << 31) >> n) << 1)` → 高 `n` 位清零（P5、P9）；
- **周期性掩码**：用 ≤ `0xFF` 的小常量移位拼接出 `0x55555555`、`0x33333333`、`0x0F0F0F0F` 等，既满足常量限制又避免超长表达式（P6、P18、P19）；
- **递推掩码**：`mask ^= mask << k` 依次生成块交换所需的掩码，比逐个手写常量更省操作数（P19）。

5.4 分治（SWAR）思想

P18 `bitCount` 与 P19 `bitReverse` 是同一思想的正反两面：把 32 位拆成 2/4/8/16 位域，逐步并行合并，`log2(32) = 5` 步完成全部工作，把线性复杂度降为对数级。二者的掩码序列互为逆序（`0x55555555 → 0x33333333 → …` 与 `0x0000FFFF → 0x00FF00FF → …`）。

5.5 IEEE 754 的统一处理

P15–P17 三个浮点函数共享同一套流程骨架：

1. 拆 `sign / exp / frac`；
2. 把 `exp == 255`（NaN/Inf）与 `±0` 单独短路，保证特殊值原样返回；
3. **规格化与非规格化统一为 `(mant, E)`**：`exp == 0` 时 `mant = frac, E = -126`，其余 `mant = (1<<23)|frac, E = exp-127`；
4. 在尾数域内用**整数运算**完成缩放或取整；
5. 用 **round-half-to-even** 舍入（`dropped > half` 进位；`dropped == half` 时看保留位最低位），并统一处理尾数进位导致的阶码加一与溢出到无穷；
6. 负数只保管 `sign` 位，所有运算在绝对值域进行，避免补码符号带来的分支。

其中"尾数进位 → 阶码加一 → 可能溢出到无穷"这一条链在 P15、P16、P17 中都出现，是浮点实现最容易遗漏的边界。

报告结束